I’m a big fan of Identity when you need to store your own users, so I tend to suggest it in most situations, as it handles a lot of security-related things for you that are easy to mess up. I’ve heard several arguments against it, some valid and others less so:

- *I already have user authentication in my app.* Great! In that case, you’re probably right, Identity may not be necessary. But does your custom implementation use MFA? Do you have account lockout? If not, and if you need to add them, considering Identity may be worthwhile.
- *I don’t want to use EF Core.* That’s a reasonable stance. You could be using Dapper, some other object-relational mapper (ORM), or even a document database for your database access. Luckily, the database integration in Identity is pluggable, so you could swap out the EF Core integration and use your own database integration libraries instead.
- *My use case is too complex for Identity.* Identity provides lower-level services for authentication, so you can compose the pieces however you like. It’s also extensible, so if you need to, for example, transform claims before creating a principal, you can.
- *I don’t like the default Razor Pages UI.* The default UI for Identity is entirely optional. You can still use the Identity services and user management but provide your own UI for logging in and registering users. However, be aware that although doing this gives you a lot of flexibility, it’s also easy to introduce a security flaw in your user-management system—the last place you want security flaws!
- *I’m not using Bootstrap to style my application.* The default Identity UI uses Bootstrap as a styling framework, the same as the default ASP.NET Core templates. Unfortunately, you can’t easily change that, so if you’re using a different framework or need to customize the HTML generated, you can still use Identity, but you’ll need to provide your own UI.